Practical 4

2.Write a program for process creation using C

●Orphan Process

●Zombie Process

```
~/process
  GNU nano 8.7                                orphan.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {

  pid_t pid = fork();

  if(pid>0){

      printf("parent process exiting\n");

  } else if (pid ==0){

      sleep(5);
      printf("Chaild process\n");
      printf("PID = %d\n", getpid());
      printf("PPID = %d\n", getppid());
  }else{
      printf("Fork failed\n");

  }

                                  [ Read 25 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

```
lenovo@LAPTOP-6627EU0U MSYS ~/process
$ Chaild process
PID = 364
PPID = 1
```

3. Create the process using fork () system call.

- Child Process creation
- Parent process creation
- PPID and PID

```
~/process
  GNU nano 8.7                                zombie.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid > 0) {
        printf("Parent process\n");
        printf("PID = %d\n", getpid());
        sleep(10);
    }
    else if (pid == 0) {
        printf("Child process exiting\n");
        printf("Child PID = %d\n", getpid());
        _exit(0);
    }
    else {
        printf("Fork failed\n");
    }

    return 0;
                                    [ Read 24 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo
```

```
lenovo@LAPTOP-6627EU0U MSYS ~/process
$ ./zombie
Parent process
Child process exiting
PID = 379
Child PID = 380
```